

CY Tech

Département Mathématiques et Informatique

ING1-GM – Année 2025–2026

Résolution d'une équation différentielle d'ordre deux

Rapport de projet

Juin 2026

Auteurs

ZERARGUI Abderrahmane
RICCO Louis
CHIBANI Redouane
BOUCHAGOUR Fares
AGBOTA Yann
NOUNEMO Franck Michel

Tuteur

MAATOUK Zahraa

Résumé

Ce rapport étudie une équation différentielle d'ordre deux qui dépend de deux paramètres μ et c . La question principale est simple : pour quelles valeurs de μ et c peut-on trouver une fonction Q définie sur $\mathbb{R}$, telle que $Q(-\infty) = c$ et $Q(+\infty) = 0$?

Autrement dit, on cherche une solution qui part d'une valeur positive c et qui descend vers 0. Pour répondre à cette question, on utilise une fonction auxiliaire

$$P(x) = \frac{-2G(x)}{x^2}.$$

L'idée du rapport est de montrer que les bons couples (c, μ) se trouvent en étudiant cette fonction P , puis de programmer cette méthode en Python.

Table des matières

1	Introduction	3
1.1	Contexte général	3
1.2	Motivations physiques	3
1.3	Objectifs du projet	3
1.4	Plan du rapport	4
2	État de l'art	4
2.1	Energie conservée et solutions reliant deux états	4
2.2	L'équation de Fisher–KPP	4
2.3	L'équation d'Allen–Cahn	5
2.4	Travaux de Berestycki–Lions	5
2.5	Méthode de tir et étude numérique	5
2.6	Stabilité et dynamique des fronts	5
3	Formulation du problème	6
3.1	Hypothèses	6
3.2	Fonctions auxiliaires	6
3.3	Énoncé du problème	6
3.4	Hypothèses	6
3.5	Fonctions auxiliaires	7
3.6	Énoncé du problème	7
3.7	L'algorithme	7
3.8	Lien avec le dessin des trajectoires	8
4	Analyse théorique	8
4.1	Partie 1 – Nécessité	8
4.2	Partie 2 – Suffisance	9
4.3	Partie 2 – Suffisance	10

5	Implémentation numérique	11
5.1	Outils utilisés	11
5.2	Calcul de G et P	11
5.3	Dérivée numérique de P	12
5.4	Recherche des paramètres	12
5.5	Résolution de l'ODE	13
5.6	Structure du code	13
6	Résultats numériques	13
6.1	Exemples testés	13
6.2	Lecture des résultats	14
6.3	Validation	14
6.4	Influence des choix numériques	14
7	Conclusion	14
7.1	Bilan	14
7.2	Influence des choix numériques	15
8	Conclusion	15
8.1	Bilan	15
8.2	Perspectives	15
	Références bibliographiques	16

1 Introduction

1.1 Contexte général

On étudie une équation différentielle d'ordre deux de la forme

$$-Q''(x) + \mu Q(x) + g(Q(x)) = 0, \quad x \in \mathbb{R}, \quad (1)$$

où $\mu > 0$ est un paramètre et où g est une fonction donnée. Le mot “non linéaire” signifie seulement que l'équation ne contient pas uniquement Q et ses dérivées de façon linéaire.

Dans ce projet, on cherche des solutions de (1) qui satisfont les conditions aux limites

$$Q(-\infty) = c > 0 \quad \text{et} \quad Q(+\infty) = 0,$$

où $c > 0$ est un second paramètre. On cherche donc une fonction Q qui relie deux valeurs : elle vaut presque c très à gauche, puis elle tend vers 0 très à droite. En mathématiques, on appelle parfois cela une connexion hétérocline, mais dans ce rapport on peut simplement penser à une solution qui relie deux états.

1.2 Motivations physiques

Ce type d'équation apparaît dans plusieurs domaines. Voici quelques exemples, expliqués simplement.

Théorie des champs scalaires. En physique, on rencontre des solutions qui relient deux états possibles d'un système. On les appelle parfois des kinks, mais on peut simplement les voir comme des fronts de transition. Un exemple classique vient de l'équation du champ ϕ^4 :

$$-\phi'' + m^2\phi - \lambda\phi^3 = 0,$$

Ces solutions modélisent un passage stable entre deux valeurs possibles du système [9].

Transitions de phase. Le modèle d'Allen–Cahn [4] sert à décrire le passage entre deux phases dans un matériau. Les profils qui ne bougent plus vérifient une équation proche de (1), par exemple avec $g(Q) = Q - Q^3$.

Biologie mathématique. L'équation de Fisher–KPP [2, 3] modélise par exemple la propagation d'une population. Le profil de propagation vérifie aussi une équation différentielle du même type.

Optique non linéaire. Dans les fibres optiques, certaines ondes gardent leur forme pendant leur déplacement. Après simplification du modèle, on retrouve aussi des équations proches de celle étudiée ici.

1.3 Objectifs du projet

Ce projet a deux objectifs principaux.

- Partie théorique.** Montrer comment trouver toutes les paires (μ, c) pour lesquelles une solution existe. On montre aussi que, quand elle existe, la solution est unique à un décalage horizontal près.
- Partie numérique.** Programmer la méthode en Python avec NumPy, SciPy et Matplotlib, puis tester le programme sur plusieurs exemples.

1.4 Plan du rapport

La section 2 donne le contexte du sujet. La section 3 présente le problème et la méthode. La section 4 contient les preuves. La section 5 explique le programme Python. La section 6 présente les résultats numériques.

2 État de l'art

2.1 Energie conservée et solutions reliant deux états

Un système hamiltonien est un système dans lequel une quantité appelée énergie reste constante. Dans le cas classique, il s'écrit

$$\dot{q} = \frac{\partial H}{\partial p}, \quad \dot{p} = -\frac{\partial H}{\partial q},$$

où $H(q, p)$ est l'énergie. La propriété importante est

$$\frac{dH}{dt} = 0.$$

Cela veut dire que l'énergie ne change pas quand la solution avance.

Notre équation peut aussi se lire comme un système avec énergie conservée :

$$Q' = P, \quad P' = \mu Q + g(Q),$$

avec l'énergie

$$H(Q, P) = \frac{P^2}{2} - G(Q) - \frac{\mu}{2}Q^2,$$

où G est une primitive de g . Une solution qui relie c à 0 correspond alors à une trajectoire qui part du point $(c, 0)$ et arrive au point $(0, 0)$.

L'idée est donc de regarder les courbes où l'énergie H garde la même valeur. Si une de ces courbes relie $(c, 0)$ à $(0, 0)$, alors on peut obtenir une solution du problème. Cette façon de voir les choses est classique dans l'étude des équations différentielles [7].

2.2 L'équation de Fisher–KPP

Un exemple connu est l'équation de Fisher–KPP, proposée par Fisher [2] puis par Kolmogorov, Petrovsky et Piskunov [3] :

$$u_t = u_{xx} + u(1 - u), \quad x \in \mathbb{R}, t \geq 0.$$

Si on cherche une solution qui se déplace sans changer de forme, on pose $u(x, t) = Q(x - ct)$. On obtient alors une équation différentielle pour le profil Q :

$$-Q'' - cQ' + Q(1 - Q) = 0.$$

Fisher et KPP ont montré qu'il existe une vitesse minimale $c^* = 2$. Pour $c \geq 2$, on trouve un front qui descend de 1 vers 0. Cet exemple montre pourquoi les profils reliant deux valeurs sont importants.

2.3 L'équation d'Allen–Cahn

Allen et Cahn [4] ont proposé un modèle pour décrire le passage entre deux phases dans un matériau :

$$u_t = \varepsilon^2 u_{xx} - W'(u),$$

Ici $W(u) = \frac{1}{4}(1-u^2)^2$ est une fonction qui a deux minimums. Les profils qui ne changent plus avec le temps vérifient $-\varepsilon^2 Q'' + W'(Q) = 0$, c'est-à-dire $-\varepsilon^2 Q'' + Q^3 - Q = 0$.

Dans ce cas, on connaît une solution explicite :

$$Q(x) = \tanh\left(\frac{x}{\varepsilon\sqrt{2}}\right),$$

Elle relie -1 à $+1$. C'est un exemple simple de front de transition.

2.4 Travaux de Berestycki–Lions

Berestycki et Lions [5] ont étudié des problèmes plus généraux sur l'existence de solutions d'énergie finie. Leur travail donne des conditions pour savoir quand une solution existe. Notre projet est plus simple, car on travaille seulement en dimension un, mais il suit la même idée générale : utiliser l'énergie pour comprendre les solutions.

Dans leurs travaux, la solution est souvent obtenue en minimisant une énergie de la forme

$$\mathcal{E}[Q] = \int_{\mathbb{R}} \left(\frac{Q'^2}{2} + F(Q) \right) dx,$$

On ne développe pas ces outils ici, mais cela montre que la notion d'énergie est centrale dans ce type de problème.

2.5 Méthode de tir et étude numérique

La méthode de tir est une méthode numérique simple pour les problèmes avec deux conditions aux limites [10]. L'idée est la suivante :

1. On choisit des valeurs de départ.
2. On résout l'équation numériquement.
3. On ajuste les valeurs de départ jusqu'à obtenir la bonne limite à l'arrivée.

Dans notre cas, l'énergie conservée simplifie beaucoup le travail. Au lieu de chercher plusieurs paramètres en même temps, on obtient la relation

$$\mu = P(c).$$

Cela transforme le problème en une recherche de bonnes valeurs de c .

2.6 Stabilité et dynamique des fronts

Une autre question importante est la stabilité. Dire qu'un front est stable signifie que, si on le modifie un peu au départ, il garde quand même sa forme globale avec le temps. Fife et McLeod [6] ont par exemple montré ce type de résultat pour Fisher–KPP.

Il existe des outils plus avancés pour étudier cela, comme l'analyse spectrale [8]. Dans ce rapport, on ne cherche pas à prouver la stabilité : on se concentre sur l'existence et le calcul des solutions.

3 Formulation du problème

3.1 Hypothèses

On se donne une fonction $g \in C^2(\mathbb{R}, \mathbb{R})$ vérifiant

$$g(0) = 0 \quad \text{et} \quad g'(0) = 0. \quad (2)$$

Ces hypothèses garantissent que $Q \equiv 0$ est un équilibre de l'équation (1), et que le développement de Taylor de g commence au second ordre : $g(t) = O(t^2)$ au voisinage de 0.

Définition 3.1. On appelle ici *solution de liaison* de c vers 0 toute solution $Q \in C^2(\mathbb{R})$ de l'équation différentielle

$$-Q''(x) + \mu Q(x) + g(Q(x)) = 0, \quad x \in \mathbb{R},$$

vérifiant $Q(-\infty) = c$ et $Q(+\infty) = 0$. Dans certains livres, ce type de solution est appelé connexion hétérocline.

3.2 Fonctions auxiliaires

Définition 3.2. On définit :

$$G(y) = \int_0^y g(t) dt \quad (y \in \mathbb{R}), \quad (3)$$

$$P(x) = -\frac{2G(x)}{x^2} \quad (x > 0). \quad (4)$$

Remarque 3.3. Puisque $g(0) = g'(0) = 0$, on a $G(x) = O(x^3)$ et donc $P(x) = O(x)$ au voisinage de 0 : P admet un prolongement continu en 0 avec $P(0) = 0$.

Par calcul direct, la dérivée de P est

$$P'(x) = \frac{2(2G(x) - xg(x))}{x^3} \quad (x > 0).$$

3.3 Énoncé du problème

Problème central

Pour quels paramètres $\mu, c > 0$ existe-t-il une solution qui part de c et arrive vers 0 pour l'équation $-Q'' + \mu Q + g(Q) = 0$?

3.4 Hypothèses

On se donne une fonction $g \in C^2(\mathbb{R}, \mathbb{R})$ vérifiant

$$g(0) = 0 \quad \text{et} \quad g'(0) = 0. \quad (5)$$

Ces hypothèses garantissent que $Q \equiv 0$ est un équilibre de l'équation (1), et que le développement de Taylor de g commence au second ordre : $g(t) = O(t^2)$ au voisinage de 0.

Définition 3.4. On appelle ici *solution de liaison* de c vers 0 toute solution $Q \in C^2(\mathbb{R})$ de l'équation différentielle

$$-Q''(x) + \mu Q(x) + g(Q(x)) = 0, \quad x \in \mathbb{R},$$

vérifiant $Q(-\infty) = c$ et $Q(+\infty) = 0$. Dans certains livres, ce type de solution est appelé connexion hétérocline.

3.5 Fonctions auxiliaires

Définition 3.5. On définit :

$$G(y) = \int_0^y g(t) dt \quad (y \in \mathbb{R}), \quad (6)$$

$$P(x) = -\frac{2G(x)}{x^2} \quad (x > 0). \quad (7)$$

Remarque 3.6. Puisque $g(0) = g'(0) = 0$, on a $G(x) = O(x^3)$ et donc $P(x) = O(x)$ au voisinage de 0 : P admet un prolongement continu en 0 avec $P(0) = 0$.

Par calcul direct, la dérivée de P est

$$P'(x) = \frac{2(2G(x) - xg(x))}{x^3} \quad (x > 0).$$

3.6 Énoncé du problème

Problème central

Pour quels paramètres $\mu, c > 0$ existe-t-il une solution qui part de c et arrive vers 0 pour l'équation $-Q'' + \mu Q + g(Q) = 0$?

3.7 L'algorithme

Algorithme de sélection des paramètres

L'algorithme revient à chercher les bons niveaux c en regardant la fonction P .

Étape 1. Calculer $P(x) = -\frac{2}{x^2} \int_0^x g(t) dt$ pour $x > 0$.

Étape 2. Trouver toutes les valeurs $c > 0$ telles que :

- $P'(c) = 0$,
- $P(c) > 0$,
- $P(x) < P(c)$ pour tout $x \in]0, c[$.

Étape 3. Pour chaque valeur c trouvée, poser $\mu = P(c)$. L'équation admet alors une solution. Cette solution est unique, sauf si on la décale horizontalement.

3.8 Lien avec le dessin des trajectoires

Dans le plan (Q, Q') , on peut dessiner les trajectoires possibles du système. La solution recherchée est une trajectoire qui relie le point $(c, 0)$ au point $(0, 0)$. Ces deux points sont des états d'équilibre du système

$$Q' = p, \quad p' = \mu Q + g(Q).$$

La condition $H \equiv 0$ signifie que la trajectoire garde la même énergie pendant tout le mouvement. Pour relier $(c, 0)$ et $(0, 0)$, ces deux points doivent donc avoir la même énergie. Cela donne $G(c) + \frac{\mu}{2}c^2 = 0$.

Les conditions sur P sont une autre manière d'écrire cette contrainte, avec en plus le fait que la solution doit descendre de c vers 0.

4 Analyse théorique

4.1 Partie 1 – Nécessité

On suppose qu'il existe une solution Q du problème pour des paramètres $\mu, c > 0$.

Dans cette partie, on part d'une solution qui existe déjà. Le but est de montrer que cette solution force les paramètres c et μ à vérifier les conditions de l'algorithme.

Proposition 4.1 (Conservation de l'énergie). *La quantité*

$$H(x) = \frac{Q'^2(x)}{2} - G(Q(x)) - \frac{\mu}{2}Q^2(x)$$

est identiquement nulle sur $\mathbb{R}$.

Démonstration. Dérivons H :

$$H'(x) = Q'(x)Q''(x) - g(Q(x))Q'(x) - \mu Q(x)Q'(x) = Q'(x)[Q''(x) - \mu Q(x) - g(Q(x))].$$

L'équation différentielle donne $Q'' = \mu Q + g(Q)$, donc $H' = 0$ et H est constante. En $+\infty$: $Q \rightarrow 0$ implique $G(Q) \rightarrow 0$ et $Q^2 \rightarrow 0$; de plus $Q'(+\infty) = 0$ car Q est monotone bornée (voir Proposition 4.5). Donc $H = H(+\infty) = 0$. $\square$

Corollaire 4.2. *Pour tout $x \in \mathbb{R}$:*

$$Q'^2(x) = \mu Q^2(x) + 2G(Q(x)). \quad (8)$$

Proposition 4.3 (Positivité de f près de 0). *Il existe $\eta > 0$ tel que $f(y) := \mu y^2 + 2G(y) > 0$ pour $y \in]0, \eta[$.*

Démonstration. Par Taylor : $g(t) = \frac{g''(0)}{2}t^2 + O(t^3)$, d'où $G(y) = \frac{g''(0)}{6}y^3 + O(y^4)$. Donc

$$f(y) = \mu y^2 + \frac{g''(0)}{3}y^3 + O(y^4) = \mu y^2 \left(1 + \frac{g''(0)}{3\mu}y + O(y^2) \right) > 0$$

pour $y > 0$ assez petit (le facteur entre parenthèses tend vers $1 > 0$). $\square$

Proposition 4.4 (f ne vaut pas zéro entre 0 et c). *La fonction $f(y) = \mu y^2 + 2G(y)$ ne s'annule pas sur $]0, c[$.*

Démonstration. On raisonne par contradiction. D'après la Proposition 4.3, $f > 0$ sur $]0, \eta[$, donc l'ensemble $\mathcal{Z} = \{y \in]0, c[: f(y) = 0\}$ est fermé dans $] \eta, c[$. Si $\mathcal{Z} \neq \emptyset$, posons $y_0 = \inf \mathcal{Z} \geq \eta > 0$. Alors $f > 0$ sur $]0, y_0[$ et $f(y_0) = 0$.

Soit x_0 tel que $Q(x_0) = y_0$. Du Corollaire 4.2 : $Q'(x_0)^2 = f(y_0) = 0$, soit $Q'(x_0) = 0$.

Le couple $(y_0, 0)$ est alors un point de départ possible pour le système. Deux cas se présentent :

Cas 1 : $\mu y_0 + g(y_0) \neq 0$. Alors $(y_0, 0)$ n'est pas un point d'équilibre. Mais comme $f(y_0) = 0$, la solution ne peut pas passer sous y_0 . C'est impossible, car on veut avoir $Q(+\infty) = 0$.

Cas 2 : $\mu y_0 + g(y_0) = 0$. Alors $(y_0, 0)$ est un point d'équilibre. Le théorème de Cauchy–Lipschitz est le résultat d'unicité pour les équations différentielles : ici, il dit qu'une solution qui part de ce point reste bloquée à cette valeur. On aurait donc $Q \equiv y_0$, ce qui contredit $Q(+\infty) = 0$.

Dans les deux cas, contradiction. $\square$

Proposition 4.5 (Q est décroissante). *Il existe $\varepsilon \in \{-1, +1\}$ constant tel que $Q'(x) = \varepsilon \sqrt{f(Q(x))}$ pour tout x , et $\varepsilon = -1$.*

Démonstration. De $Q'^2 = f(Q)$ et $f(Q(x)) > 0$ pour $Q(x) \in]0, c[$, on tire $Q'(x) = \varepsilon(x) \sqrt{f(Q(x))}$ avec $\varepsilon(x) \in \{-1, +1\}$. Comme Q' est continue et $\sqrt{f(Q)}$ ne s'annule pas, ε est continue à valeurs dans $\{-1, +1\}$, donc constante. Si $\varepsilon = +1$, Q est croissante, ce qui contredit $Q(-\infty) = c > 0 = Q(+\infty)$. Donc $\varepsilon = -1$ et Q est strictement décroissante. $\square$

Proposition 4.6 (Condition de raccordement). *On a $\mu c + g(c) = 0$, d'où $g(c) = -\mu c < 0$.*

Démonstration. Puisque Q est décroissante et bornée par 0, Q' est bornée et admet une limite en $-\infty$: $Q'(-\infty) = 0$. Du Corollaire 4.2 en $-\infty$:

$$0 = f(c) = \mu c^2 + 2G(c). \quad (9)$$

L'ODE donne $Q'' = \mu Q + g(Q)$. En $-\infty$, si $\mu c + g(c) = L \neq 0$, alors $Q''(x) \rightarrow L$, d'où $Q'(x) \approx Lx$ qui diverge, contradiction. Donc $L = 0$, soit $g(c) = -\mu c < 0$. $\square$

Théorème 4.7 (Nécessité). *Si le problème admet une solution, alors c satisfait les conditions de l'algorithme avec $\mu = P(c)$.*

Démonstration. $\mu = P(c)$: De (9), $\mu = -2G(c)/c^2 = P(c) > 0$.

$P'(c) = 0$: $P'(x) = 2(2G(x) - xg(x))/x^3$. Avec $G(c) = -\mu c^2/2$ et $g(c) = -\mu c$: $P'(c) = 2(2 \cdot (-\mu c^2/2) - c(-\mu c))/c^3 = 2(-\mu c^2 + \mu c^2)/c^3 = 0$.

$P(x) < P(c)$ sur $]0, c[$: Pour $x \in]0, c[$, cela équivaut à $f(x) = \mu x^2 + 2G(x) > 0$, qui est la Proposition 4.4. $\square$

4.2 Partie 2 – Suffisance

On suppose que $c, \mu > 0$ satisfont les conditions de l'algorithme.

Ici, on fait le raisonnement inverse. On part d'une valeur c trouvée par l'algorithme, puis on construit une solution Q .

Proposition 4.8. *La fonction $f(y) = \mu y^2 + 2G(y)$ est strictement positive sur $]0, c[$.*

Démonstration. Pour $y \in]0, c[$: $P(y) < P(c) = \mu$, soit $-2G(y)/y^2 < \mu$, soit $f(y) = \mu y^2 + 2G(y) > 0$. $\square$

Théorème 4.9 (Existence). *Sous les hypothèses de l'algorithme, le problème admet une solution.*

Démonstration. Définissons, pour $y_0 \in]0, c[$ fixé,

L'ODE donne $Q'' = \mu Q + g(Q)$. En $-\infty$, si $\mu c + g(c) = L \neq 0$, alors $Q''(x) \rightarrow L$, d'où $Q'(x) \approx Lx$ qui diverge, contradiction. Donc $L = 0$, soit $g(c) = -\mu c < 0$. $\square$

Théorème 4.10 (Nécessité). *Si le problème admet une solution, alors c satisfait les conditions de l'algorithme avec $\mu = P(c)$.*

Démonstration. $\mu = P(c)$: De (9), $\mu = -2G(c)/c^2 = P(c) > 0$.

$P'(c) = 0$: $P'(x) = 2(2G(x) - xg(x))/x^3$. Avec $G(c) = -\mu c^2/2$ et $g(c) = -\mu c$: $P'(c) = 2(2 \cdot (-\mu c^2/2) - c(-\mu c))/c^3 = 2(-\mu c^2 + \mu c^2)/c^3 = 0$.

$P(x) < P(c)$ sur $]0, c[$: Pour $x \in]0, c[$, cela équivaut à $f(x) = \mu x^2 + 2G(x) > 0$, qui est la Proposition 4.4. $\square$

4.3 Partie 2 – Suffisance

On suppose que $c, \mu > 0$ satisfont les conditions de l'algorithme.

Ici, on fait le raisonnement inverse. On part d'une valeur c trouvée par l'algorithme, puis on construit une solution Q .

Proposition 4.11. *La fonction $f(y) = \mu y^2 + 2G(y)$ est strictement positive sur $]0, c[$.*

Démonstration. Pour $y \in]0, c[$: $P(y) < P(c) = \mu$, soit $-2G(y)/y^2 < \mu$, soit $f(y) = \mu y^2 + 2G(y) > 0$. $\square$

Théorème 4.12 (Existence). *Sous les hypothèses de l'algorithme, le problème admet une solution.*

Démonstration. Définissons, pour $y_0 \in]0, c[$ fixé,

$$\Phi(y) = \int_y^{y_0} \frac{ds}{\sqrt{f(s)}}, \quad y \in]0, c[.$$

L'intégrande est positif et continu sur $]0, c[$ (Proposition 4.11).

Comportement près de 0 : $f(s) \sim \mu s^2$ d'après la Proposition 4.3, donc $\frac{1}{\sqrt{f(s)}} \sim \frac{1}{s\sqrt{\mu}}$ et $\int_0^{y_0} \frac{ds}{s} = +\infty$. Ainsi $\Phi(y) \rightarrow +\infty$ quand $y \rightarrow 0^+$.

Comportement près de c : $f(c) = 0$ (de $\mu = P(c)$) et $f'(c) = 2\mu c + 2g(c) = 2(\mu c + g(c))$. La condition $P'(c) = 0$ équivaut à $g(c) = -\mu c$, d'où $f'(c) = 0$. Donc $f(s) = O((s - c)^2)$ et $\int_{y_0}^c \frac{ds}{\sqrt{f(s)}}$ diverge. Ainsi $\Phi(y) \rightarrow -\infty$ quand $y \rightarrow c^-$.

Donc Φ associe les valeurs de $]0, c[$ à tous les réels. On peut alors définir Q comme la fonction inverse de Φ . Par dérivation de la fonction inverse :

$$Q'(x) = \frac{1}{\Phi'(Q(x))} = \frac{1}{-1/\sqrt{f(Q(x))}} = -\sqrt{f(Q(x))}.$$

En dérivant une seconde fois :

$$Q''(x) = -\frac{f'(Q)Q'}{2\sqrt{f(Q)}} = -\frac{(2\mu Q + 2g(Q))(-\sqrt{f(Q)})}{2\sqrt{f(Q)}} = \mu Q + g(Q).$$

Donc $-Q'' + \mu Q + g(Q) = 0$. Les limites $Q(-\infty) = c$ et $Q(+\infty) = 0$ découlent des divergences de Φ . $\square$

Théorème 4.13 (Unicité à décalage horizontal près). *Si $\tilde{Q}$ est une autre solution, il existe $a \in \mathbb{R}$ tel que $\tilde{Q}(x) = Q(x + a)$ pour tout $x \in \mathbb{R}$.*

Démonstration. Par la Partie 1, $\tilde{Q}$ satisfait aussi $\tilde{Q}' = -\sqrt{f(\tilde{Q})}$. Comme Q est une bijection continue de $\mathbb{R}$ sur $]0, c[$, il existe x_1 tel que $Q(x_1) = \tilde{Q}(0)$. Alors $Q(x_1) = \tilde{Q}(0)$ et $Q'(x_1) = -\sqrt{f(Q(x_1))} = -\sqrt{f(\tilde{Q}(0))} = \tilde{Q}'(0)$. Par le même résultat d'unicité, les deux solutions sont ensuite les mêmes après ce décalage :

$$\tilde{Q}(x) = Q(x + x_1).$$

$\square$

5 Implémentation numérique

5.1 Outils utilisés

L'implémentation utilise Python 3 avec les bibliothèques suivantes :

- **NumPy** [13] : tableaux numériques, fonctions rapides sur des listes de nombres.
- **SciPy** [12] : intégration numérique (`integrate.quad`, `integrate.solve_ivp`) et recherche de racines (`optimize.brentq`).
- **Matplotlib** [14] : tracé des graphes.

5.2 Calcul de G et P

```

1 def make_G(g):
2     """G(y) = integral of g from 0 to y."""
3     def G(y):
4         val, _ = integrate.quad(g, 0.0, y, limit=200)
5         return val
6     return G
7
8 def make_P(g):
9     """P(x) = -2*G(x)/x^2 for x > 0."""
10    def P(x):
11        if x <= 0:
12            return np.nan
13        val, _ = integrate.quad(g, 0.0, x, limit=200)
14        return -2.0 * val / x**2
15    return P

```

Listing 1 – Construction des fonctions G et P

La fonction `scipy.integrate.quad` calcule les intégrales numériquement avec une bonne précision. Le paramètre `limit=200` donne assez de marge lorsque la fonction est plus difficile à intégrer.

5.3 Dérivée numérique de P

```

1 def dP_numerical(P, x, h=1e-5):
2     """Centered finite difference approximation of P'(x)."""
3     return (P(x + h) - P(x - h)) / (2.0 * h)

```

Listing 2 – Dérivée numérique de P par différence centrée

Le pas $h = 10^{-5}$ donne un bon compromis :

- si h est trop grand, l'approximation est trop grossière ;
- si h est trop petit, les erreurs d'arrondi deviennent visibles.

Ici, l'erreur totale est de l'ordre de

$$10^{-10},$$

ce qui est suffisant pour le projet.

5.4 Recherche des paramètres

```

1 def find_parameters(g, x_max=10.0, n_grid=2000):
2     P = make_P(g)
3     # Step 1: dense grid scan for sign changes of P'
4     xs = np.linspace(1e-3, x_max, n_grid)
5     dP_vals = np.array([dP_numerical(P, x) for x in xs])
6     sign_changes = np.where(np.diff(np.sign(dP_vals)))[0]
7     candidates = []
8     for idx in sign_changes:
9         # Step 2: Brent refinement
10        try:
11            c_star = optimize.brentq(
12                lambda x: dP_numerical(P, x),
13                xs[idx], xs[idx + 1], xtol=1e-10)
14        except Exception:
15            continue
16        mu_star = P(c_star)
17        if mu_star <= 0.0:
18            continue
19        # Step 3: global maximum check
20        x_check = np.linspace(1e-4, c_star - 1e-4, 500)
21        P_check = np.array([P(x) for x in x_check])
22        if np.all(P_check < mu_star - 1e-10):
23            candidates.append((float(c_star), float(mu_star)))
24    return candidates

```

Listing 3 – Algorithme de recherche des valeurs de c

Coût du calcul. Le temps de calcul dépend surtout du nombre de points de la grille `n_grid`. Plus la grille est fine, plus on détecte bien les candidats, mais plus le calcul prend du temps. En pratique, avec `n_grid = 2000`, le calcul reste rapide.

5.5 Résolution de l'ODE

```

1 def f_rhs(x, state, mu, G):
2     """Right-hand side: Q' = -sqrt(mu*Q^2 + 2*G(Q))."""
3     Q = np.clip(state[0], 1e-12, None)
4     val = max(mu * Q**2 + 2.0 * G(Q), 0.0)
5     return [-np.sqrt(val)]
6
7 sol = integrate.solve_ivp(
8     fun=lambda x, s: f_rhs(x, s, mu, G),
9     t_span=(-15.0, 15.0),
10    y0=[c / 2.0],          # initial condition Q(0) = c/2
11    method='RK45',
12    max_step=0.05,
13    rtol=1e-8,
14    atol=1e-10)

```

Listing 4 – Résolution de $Q' = -\sqrt{f(Q)}$ par RK45

La méthode RK45 [11] est une méthode classique pour résoudre numériquement une équation différentielle. Les tolérances `rtol=1e-8` et `atol=1e-10` demandent au solveur une bonne précision. On choisit $Q(0) = c/2$ simplement pour commencer au milieu entre 0 et c .

Le fait de forcer Q à rester au-dessus de 10^{-12} est une précaution numérique. Près de 0, la racine carrée devient difficile à calculer proprement. Cette précaution évite donc des erreurs inutiles.

5.6 Structure du code

Le code est organisé en quatre fonctions principales :

Fonction	Rôle
<code>make_G(g)</code>	Calcule la primitive $G = \int_0^y g$
<code>make_P(g)</code>	Calcule $P(x) = -2G(x)/x^2$
<code>find_parameters(g)</code>	Trouve tous les couples (c, μ) valides
<code>solve_and_plot(g, c, mu)</code>	Résout l'équation et trace Q
<code>visualize_all(g)</code>	Lance tout le calcul et affiche les graphes

6 Résultats numériques

6.1 Exemples testés

Nous avons appliqué l'algorithme à quatre fonctions g satisfaisant les hypothèses $g(0) = g'(0) = 0$, dont les résultats sont résumés dans le Tableau 1.

La colonne $\mu c + g(c)$ vérifie la condition de raccordement de la Proposition 4.6 : les erreurs sont inférieures à 10^{-8} , ce qui confirme la cohérence des résultats.

TABLE 1 – Paramètres obtenus pour différentes fonctions g

Ex.	$g(t)$	c	$\mu = P(c)$	$\mu c + g(c)$
1	$t^3 - t^2$	0.66667	0.22222	$< 10^{-8}$
2	$t^2(t - 2)$	1.33333	0.88889	$< 10^{-8}$
3	$t^2 \sin(t)$	5.91731	2.11703	$< 10^{-8}$
4	$t^3 - 2t^2$	1.33333	0.88889	$< 10^{-8}$

6.2 Lecture des résultats

Exemples 2 et 4. Les fonctions $g_2(t) = t^2(t - 2)$ et $g_4(t) = t^3 - 2t^2$ sont identiques, d'où des résultats identiques.

Exemple 3. La valeur $c \approx 5.917$ est nettement plus grande que dans les autres exemples. Cela vient du terme $\sin(t)$, qui fait varier la fonction plus fortement. Le maximum utile de P arrive donc plus loin, et la solution obtenue est plus étalée.

Forme des solutions. Toutes les solutions obtenues ont la même forme générale : Q descend progressivement de c vers 0, sans remonter. Cela correspond bien à ce qui a été prouvé dans la partie théorique.

6.3 Validation

Pour chaque exemple, on vérifie trois conditions après le calcul :

- (i) $|\mu c + g(c)| < 10^{-8}$: la condition au bord gauche est bien respectée.
- (ii) $|\mu c^2 + 2G(c)| < 10^{-8}$: l'énergie est bien conservée.
- (iii) $|Q(\pm 15) - Q(\pm \infty)| < 10^{-4}$: convergence numérique vers les limites.

Ces trois tests sont validés pour tous les exemples. Cela montre que le programme donne des résultats cohérents.

6.4 Influence des choix numériques

Nous avons aussi regardé l'effet de certains choix numériques :

- **Nombre de points de la grille n .** Pour $n \geq 500$, tous les candidats sont détectés. Pour $n < 100$, certains candidats peuvent être manqués si P' change de signe sur un interval court.
- **Borne $x_{\max}$.** Si $c > x_{\max}$, le candidat n'est pas trouvé. On recommande $x_{\max} = 10$ pour les exemples standards, et on peut augmenter si nécessaire.
- **Précision demandée à Brent.** Avec une précision de 10^{-10} , la valeur de c est très bien approchée. Cette précision est largement suffisante pour tracer les graphes.

7 Conclusion

7.1 Bilan

Ce projet a permis d'étudier l'existence et l'unicité des solutions qui relient c à 0 pour l'équation $-Q'' + \mu Q + g(Q) = 0$. Les résultats principaux obtenus sont :

1. Rigueur théorique. L'algorithme est justifié mathématiquement :

- Si une solution existe, alors elle donne une valeur c qui vérifie les conditions de l'algorithme (Théorème 4.10).
- Si une valeur c vérifie ces conditions, alors on peut construire une solution (Théorème 4.12).
- La solution est unique à décalage horizontal près (Théorème 4.13).

2. Efficacité numérique. Le programme Python trouve les bons couples (c, μ) et trace les

Ces trois tests sont validés pour tous les exemples. Cela montre que le programme donne des résultats cohérents.

7.2 Influence des choix numériques

Nous avons aussi regardé l'effet de certains choix numériques :

- **Nombre de points de la grille n .** Pour $n \geq 500$, tous les candidats sont détectés. Pour $n < 100$, certains candidats peuvent être manqués si P' change de signe sur un interval court.
- **Borne $x_{\max}$.** Si $c > x_{\max}$, le candidat n'est pas trouvé. On recommande $x_{\max} = 10$ pour les exemples standards, et on peut augmenter si nécessaire.
- **Précision demandée à Brent.** Avec une précision de 10^{-10} , la valeur de c est très bien approchée. Cette précision est largement suffisante pour tracer les graphes.

8 Conclusion

8.1 Bilan

Ce projet a permis d'étudier l'existence et l'unicité des solutions qui relient c à 0 pour l'équation $-Q'' + \mu Q + g(Q) = 0$. Les résultats principaux obtenus sont :

1. Rigueur théorique. L'algorithme est justifié mathématiquement :

- Si une solution existe, alors elle donne une valeur c qui vérifie les conditions de l'algorithme (Théorème 4.10).
- Si une valeur c vérifie ces conditions, alors on peut construire une solution (Théorème 4.12).
- La solution est unique à décalage horizontal près (Théorème 4.13).

2. Efficacité numérique. Le programme Python trouve les bons couples (c, μ) et trace les solutions. Les tests numériques confirment que les résultats sont corrects sur les exemples choisis.

8.2 Perspectives

On pourrait prolonger ce travail de plusieurs façons :

- **Stabilité des fronts.** Étudier si les fronts restent proches de leur forme initiale quand on les perturbe un peu.
- **Autres types d'équations.** Généraliser à des équations avec des opérateurs fractionnaires $-(-\Delta)^s Q + \mu Q + g(Q) = 0$ avec $s \in (0, 1)$.

- **Dimensions plus grandes.** Regarder ce qui se passe pour des solutions radiales dans $\mathbb{R}^N$: $-Q'' - \frac{N-1}{r}Q' + \mu Q + g(Q) = 0$.
- **Plusieurs solutions possibles.** Chercher si certaines fonctions g peuvent donner plusieurs profils différents.

Références bibliographiques

Références

- [1] H. Poincaré, *Les méthodes nouvelles de la mécanique céleste*, Gauthier-Villars, Paris, 1892.
- [2] R. A. Fisher, “The wave of advance of advantageous genes,” *Annals of Eugenics*, vol. 7, pp. 355–369, 1937.
- [3] A. N. Kolmogorov, I. G. Petrovsky, and N. S. Piskunov, “Étude de l’équation de la diffusion avec croissance de la quantité de matière et son application à un problème biologique,” *Bull. Univ. Moscow, Ser. Internat., Sect. A*, vol. 1, pp. 1–25, 1937.
- [4] S. M. Allen and J. W. Cahn, “A microscopic theory for antiphase boundary motion and its application to antiphase domain coarsening,” *Acta Metallurgica*, vol. 27, no. 6, pp. 1085–1095, 1979.
- [5] H. Berestycki and P.-L. Lions, “Nonlinear scalar field equations I : Existence of a ground state,” *Archive for Rational Mechanics and Analysis*, vol. 82, no. 4, pp. 313–345, 1983.
- [6] P. C. Fife and J. B. McLeod, “The approach of solutions of nonlinear diffusion equations to travelling front solutions,” *Archive for Rational Mechanics and Analysis*, vol. 65, no. 4, pp. 335–361, 1977.
- [7] P. H. Rabinowitz, “Heteroclinics for a reversible Hamiltonian system,” *Ergodic Theory and Dynamical Systems*, vol. 14, pp. 817–829, 1994.
- [8] D. H. Sattinger, “On the stability of waves of nonlinear parabolic systems,” *Advances in Mathematics*, vol. 22, pp. 312–355, 1976.
- [9] R. Rajaraman, *Solitons and Instantons*, North-Holland, Amsterdam, 1982.
- [10] H. B. Keller, *Numerical Solution of Two Point Boundary Value Problems*, SIAM, Philadelphia, 1976.
- [11] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I : Nonstiff Problems*, 2nd ed., Springer-Verlag, Berlin, 1993.
- [12] P. Virtanen et al., “SciPy 1.0 : Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [13] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [14] J. D. Hunter, “Matplotlib : A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.